

Sobolev Training for Option Pricing Surrogates

Software Lab Computational Engineering Science
Final Report

Group 7

Yan Liu
Marin Zhuka
Karl Holtmannspötter
Simon Rings
Ole Stappen

Supervisor: Neil Kichler
Informatik 12: Software and Tools for Computational Engineering (STCE)
RWTH Aachen University

August 2026

Contents

1	Analysis	7
1.1	User Requirements	7
1.2	System Requirements	7
2	Design	9
2.1	Principal Components and Third-Party Software	9
2.2	Class Models	9
3	Implementation	11
3.1	Development Infrastructure	11
3.2	Source Code	11
3.3	Kalibrierung	11
3.4	Visualisierung und Reporting	12
3.5	Software Tests	13
4	Project Management	15
A	User Documentation	19
A.1	Building	19
A.2	Testing	19
A.3	Running	19

Preface

- administrative information about the project (e.g, topic issued by which institute)
- fit of topic into study program (e.g, sufficient prior knowledge)
- acknowledgement of supervision

Chapter 1

Analysis

1.1 User Requirements

user requirements explained (includes essential information and references into literature on technical background of the topic, e.g, [1]) based on UML Use Case diagram(s)

1.2 System Requirements

functional and non-functional system requirements explained

Chapter 2

Design

2.1 Principal Components and Third-Party Software

libraries that you built on explained briefly and references to further information

2.2 Class Models

UML Class diagram(s) and description; should link into overall design through reference of application programming interfaces (API) of third-party software

Chapter 3

Implementation

3.1 Development Infrastructure

Wir haben das Projekt in Python umgesetzt. Als Grundlage nutzen wir JAX für die numerischen Berechnungen, Equinox für die Modellkomponenten und Optimistix für die Kalibrierung. Für das Laden der Marktdaten verwenden wir zusätzlich `yfinance`.

3.2 Source Code

Der Code ist in mehrere Pakete aufgeteilt. `kalibrierung` enthält das Laden der Marktdaten und die Kalibrierung, `marktsimulation` die Preisfunktionen und `surrogate_modeling` das Sobolev-Training. Die Risikoberechnung liegt in `risk_visualisierung`.

Die zentrale Einstiegsstelle ist `main.py`. Dort werden die Schritte nacheinander ausgeführt: Marktdaten laden, kalibrieren, Dataset erzeugen, Surrogat trainieren und Ergebnisse auswerten. Das hält den Ablauf übersichtlich und macht Debugging einfacher.

3.3 Kalibrierung

Die Kalibrierung ist der erste praktische Schritt unserer Pipeline. Dabei werden echte Marktdaten mit dem Black-Scholes-Modell verglichen, damit die Modellparameter sinnvoll angepasst werden können.

Die Marktdaten werden über den `MarketDataLoader` aus Yahoo Finance geladen. Danach werden nur liquide Optionen behalten, also Kontrakte mit positivem Bid/Ask, genügend Volumen und Open Interest. Aus Bid und Ask wird ein Mid Price gebildet.

Die Daten werden in `MarketData` gesammelt. Dort sind Spot, Strikes,

Laufzeiten, Marktpreise, Calls/Puts und Gewichte zusammengefasst. Außerdem prüft die Klasse direkt, ob die Arrays zusammenpassen.

Die eigentliche Optimierung übernimmt **Calibrator**. Dabei werden die Modellpreise mit den Marktpreisen verglichen und mit Levenberg-Marquardt minimiert. In `main.py` wird dafür eine kleine Wrapper-Funktion verwendet, die das Black-Scholes-Modell mit dem aktuellen Spot verbindet. Als Startwert nehmen wir eine erste Schätzung für `sigma`.

Die Kalibrierung ist nicht als einzelner Schritt gedacht, sondern als Startpunkt für den Rest der Pipeline. Erst danach werden die Sobolev-Daten erzeugt, das Surrogat trainiert und die Risikoanalyse ausgeführt. So bauen alle Teile auf denselben Modellparametern auf.

3.4 Visualisierung und Reporting

Das Modul `risk_visualisierung` ist in die drei Komponenten `xva_analysis`, `riskengine` und `visualizer` unterteilt und verbindet Surrogat-Evaluation mit Monte-Carlo-Simulationen und XVA-Berechnungen.

Die Visualisierung der Ergebnisse und Trainingsverläufe wird durch die Klasse **Visualizer** übernommen. Diese verwendet `matplotlib` für die Erstellung der Graphen und nutzt standardmäßig das `Agg`-Backend, falls keine direkte Bildschirmausgabe gefordert ist. Alle generierten Graphen können lokal oder über einen Logger gespeichert werden.

Zu den Hauptfunktionen der **Visualizer**-Klasse gehören:

- **Trainingsdiagnostik:** `plot_training_history` visualisiert den Gesamtverlust des Sobolev-Trainings sowie den Price-RMSE; Verlustkomponenten (Preis, Gradienten, HVP) werden separat gezeigt.
- **Surrogat-Evaluierung:** `plot_price_comparison` vergleicht Surrogat-Preise mit Ground-Truth; `plot_surrogate_surface` erzeugt 3D-Oberflächen zur Inspektion von Eingabe-Einflussgrößen.
- **Risikovisualisierung:** `plot_exposure_profiles` plottet EPE (Expected Positive Exposure) und ENE (Expected Negative Exposure); `plot_mc_paths` visualisiert generierte Monte-Carlo-Pfade.
- **Reporting:** `report_risk_metrics` und `report_metrics` geben berechnete Kennzahlen (z.B. CVA, DVA, NetXVA) formatiert in der Konsole aus.

Das Modul `xva_analysis` stellt den Ablauf zur XVA-Validierung bereit. Die Funktion `run_xva_analysis` erzeugt für ein gegebenes `PricingProblem` Monte-Carlo-Feature-Pfade (über `exposure_paths`), wertet diese über das Surrogat oder eine geschlossene Referenzfunktion aus und berechnet für jede betrachtete Moneyess-Stufe Exposure-Profile sowie XVA-Kennzahlen. Gibt es eine analytischen Referenz werden Surrogat- und Referenz-XVA verglichen und der

relative Fehler ausgegeben. Ergebnisvisualisierungen und Reports werden über mit Funktionen der Klasse **Visualizer** erzeugt.

Die Klasse **RiskEngine** enthält die eigentlichen Routinen zur Bestimmung von Exposure-Profilen und XVA. Parameter wie **recovery_rate** (Standard: 0.4) und **hazard_rate** (Standard: 0.02) legen dabei die Loss-Given-Default (LGD) und die zugrunde liegenden Ausfallwahrscheinlichkeiten fest. Es gibt zwei wichtige Funktionen: `compute_xva_risk`, das ein trainiertes Surrogat entgegennimmt, und `compute_xva_risk_reference`, das eine beliebige Bewertungsfunktion (z.B. `surrogate.predict_price` oder eine analytische Formel) vektorisiert auswertet. Aus der parametrischen Überlebensfunktion (mit **hazard_rate**) werden diskrete Default-Wahrscheinlichkeiten abgeleitet, mit denen die Exposure-Profile gewichtet werden; die CVA bzw. DVA ergeben sich dann als $LGD * \sum(EPE * default_probs)$ bzw. $LGD * \sum(ENE * default_probs)$, und **NetXVA** ist die Differenz $CVA - DVA$. Die Riskengine erstellt auch die zugrundeliegende **V_matrix**, sodass sich anschließende Visualisierungen (z.B. EPE/ENE-Plots) und Vergleiche mit Referenzwerten leicht durchführen lassen.

Wir trennen so Simulation (**PricingProblem**), Bewertungslogik (Surrogat oder Referenz), Risk-Formeln (**RiskEngine**) und Visualisierung/Reporting (**Visualizer**), um eine saubere Struktur zu erhalten.

3.5 Software Tests

Für die Tests haben wir vor allem auf Plausibilität und einen durchlaufenden Gesamtworkflow geachtet. Wichtig war, dass die Marktdaten geladen werden können, die Kalibrierung läuft und am Ende sinnvolle Parameter zurückkommen.

Zusätzlich haben wir die komplette Pipeline mit realen Marktsnapshots ausgeführt. So konnten wir prüfen, dass die einzelnen Module auch zusammen sauber funktionieren.

Chapter 4

Project Management

who did what, when, and why; organization of collaboration, i.e. [online] meetings, software version control (e.g, git¹

¹<https://git.rwth-aachen.de>

Bibliography

- [1] Adam Ries. *Rechenung auff der Linihen und Federn*. Annaberg, 1522.

Appendix A

User Documentation

A.1 Building

e.g, using `cmake`¹ and `make`²

A.2 Testing

e.g, `make test`

A.3 Running

documented sample session(s); e.g, `make run`

¹<https://cmake.org/>

²<https://www.gnu.org/software/make/>